

Faltungsformel für zwei unabhängige diskrete Zufallsvariablen

- Sei $Z = X + Y$, wobei X und Y zwei unabhängige diskrete Zufallsvariablen sind

$$\text{Dann ist } f_Z(z) = \sum_{x \in W_X} f_X(x) f_Y(z-x)$$

Wahrscheinlichkeit dass die Summe den Wert z hat

über alle möglichen Werte für x summieren

damit die Summe z ist, muss Y folglich den Restwert $z-x$ annehmen, weil X den Wert x hat. $x + \underbrace{(z-x)}_{=y} = z$

Zwei faire, vierseitige Würfel werden geworfen. Die Zufallsvariablen X und Y geben die Augenzahlen an. Das heisst, für alle x bzw. y im Wertebereich $W_X = W_Y = \{1, 2, 3, 4\}$ ist $f_X(x) = \frac{1}{4}$ bzw. $f_Y(y) = \frac{1}{4}$. Wir definieren $Z = X + Y$ als die Summe der Augenzahlen.

Berechne $f_Z(3)$ und $f_Z(5)$ mit der Faltungsformel.

$$\begin{aligned} f_Z(3) &= \sum_{x \in W_X} f_X(x) \cdot f_Y(3-x) \\ &= f_X(1) f_Y(3-1) + f_X(2) f_Y(3-2) + f_X(3) f_Y(3-3) + f_X(4) f_Y(3-4) \\ &= f_X(1) f_Y(2) + f_X(2) f_Y(1) + f_X(3) f_Y(0) + f_X(4) f_Y(-1) \\ &= \frac{1}{4} \frac{1}{4} + \frac{1}{4} \frac{1}{4} + \frac{1}{4} 0 + \frac{1}{4} \cdot 0 \\ &= \underline{\underline{\frac{1}{8}}} \end{aligned}$$

$$\begin{aligned} f_Z(5) &= \sum_{x \in W_X} f_X(x) \cdot f_Y(5-x) \\ &= f_X(1) f_Y(5-1) + f_X(2) f_Y(5-2) + f_X(3) f_Y(5-3) + f_X(4) f_Y(5-4) \\ &= f_X(1) f_Y(4) + f_X(2) f_Y(3) + f_X(3) f_Y(2) + f_X(4) f_Y(1) \\ &= \frac{1}{4} \frac{1}{4} + \frac{1}{4} \frac{1}{4} + \frac{1}{4} \frac{1}{4} + \frac{1}{4} \frac{1}{4} \\ &= \underline{\underline{\frac{1}{4}}} \end{aligned}$$

Gemeinsame Dichte: $f_{X,Y}(x,y) \stackrel{\text{def}}{=} \Pr[X=x, Y=y]$

Randdichte: $f_X(x) \stackrel{\text{def}}{=} \Pr[X=x]$

$$f_Y(y) \stackrel{\text{def}}{=} \Pr[Y=y]$$

Zusammenhang: $f_X(x) = \sum_{y \in W_Y} f_{X,Y}(x,y)$

Summe über alle möglichen y ,
d.h. der Einfluss von y fällt weg

		X		
		1	2	3
Y	4	0.1	0	0.1
	6	0.2	0.25	0
	9	0.1	0.15	0.1

$\Pr[X=1, Y=9]$

$$\begin{aligned}
 f_X(2) &= \sum_{y \in W_Y} f_{X,Y}(2,y) \\
 &= f_{X,Y}(2,4) + f_{X,Y}(2,6) + f_{X,Y}(2,9) \\
 &= 0 + 0.25 + 0.15 \\
 &= \underline{\underline{0.4}}
 \end{aligned}$$

Gemeinsame Verteilung: $F_{X,Y}(x,y) \stackrel{\text{def}}{=} \Pr[X \leq x, Y \leq y]$

Randverteilung: $F_X(x) \stackrel{\text{def}}{=} \Pr[X \leq x]$

$$F_Y(y) \stackrel{\text{def}}{=} \Pr[Y \leq y]$$

Zusammenhang: $F_{X,Y}(x,y) = \sum_{\substack{x' \in W_X \\ x' \leq x}} \sum_{\substack{y' \in W_Y \\ y' \leq y}} f_{X,Y}(x',y')$

		X		
		1	2	3
Y	4	0.1	0	0.1
	6	0.2	0.25	0
	9	0.1	0.15	0.1

$$\begin{aligned}
 F_{X,Y}(2,6) &= \sum_{\substack{x' \in W_X \\ x' \leq 2}} \sum_{\substack{y' \in W_Y \\ y' \leq 6}} f_{X,Y}(x',y') \\
 &= f_{X,Y}(1,4) + f_{X,Y}(1,6) + f_{X,Y}(2,4) + f_{X,Y}(2,6) \\
 &= 0.1 + 0.2 + 0 + 0.25 \\
 &= \underline{\underline{0.55}}
 \end{aligned}$$

Waldsche Identität

- Seien N und X zwei unabhängige Zufallsvariablen mit $W_N \subseteq \mathbb{N}$, und sei $Z = \sum_{i=1}^N X_i$ die Summe von N unabhängigen Kopien von X .

Dann ist $\mathbb{E}[Z] = \mathbb{E}[N] \cdot \mathbb{E}[X]$

Eine Autoversicherung analysiert die täglichen Schadensmeldungen. Die Anzahl der gemeldeten Unfälle pro Tag wird durch eine Zufallsvariable N beschrieben, die Poisson-verteilt ist mit dem Parameter $\lambda = 5$.

Die Kosten eines einzelnen Unfalls werden durch die Zufallsvariable X repräsentiert. Dabei gibt es zwei Kategorien von Schäden:

- **Totalschaden:** Kosten von 10.000 \$ mit einer Wahrscheinlichkeit von 10%.
- **Teilschaden:** Kosten von 2.000 \$ mit der restlichen Wahrscheinlichkeit 90%.

Berechne unter Verwendung der **Waldschen Identität** die erwarteten Gesamtkosten $\mathbb{E}[Z]$ pro Tag.

$$\mathbb{E}[N] = 5 \quad (\text{weil } N \text{ Poisson-verteilt ist})$$

$$\mathbb{E}[X] = \mathbb{E}[X | \text{Totalschaden}] \cdot \Pr[\text{Totalschaden}] + \mathbb{E}[X | \text{Teilschaden}] \cdot \Pr[\text{Teilschaden}] \quad (\text{Satz für Code Expert!})$$

$$= 10'000 \cdot 0,1 + 2'000 \cdot 0,9$$

$$= 1000 + 1'800$$

$$= 2'800$$

$$\mathbb{E}[Z] = \mathbb{E}[N] \cdot \mathbb{E}[X]$$

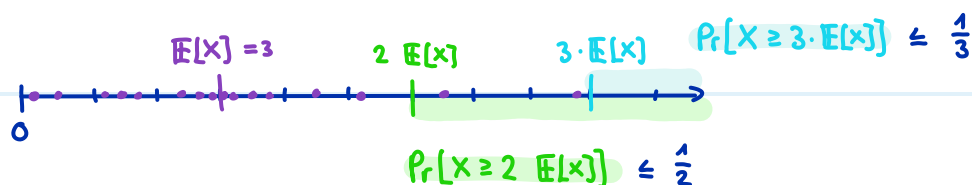
$$= 5 \cdot 2'800$$

$$= \underline{\underline{14'000}}$$

Abschätzungen

Markov: Für eine nicht-negative Zufallsvariable X und ein beliebiges $t \in \mathbb{R}$ mit $t > 0$

gilt: $\Pr[X \geq t] \leq \frac{\mathbb{E}[X]}{t}$ oder anders umgeformt: $\Pr[X \geq t \cdot \mathbb{E}[X]] \leq \frac{1}{t}$

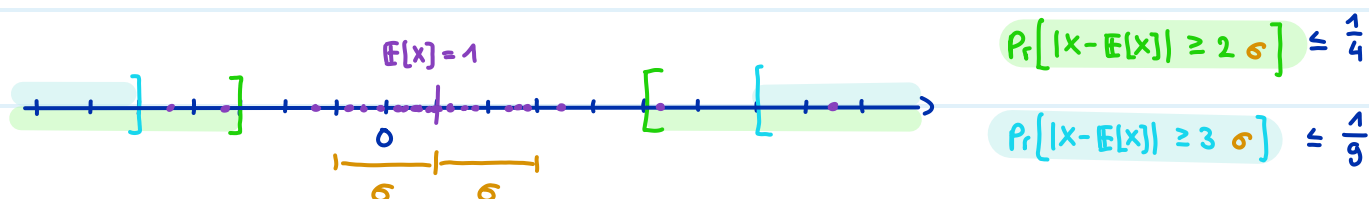


Intuitiv: Es ist unwahrscheinlich, dass die Werte von X viel grösser als der Erwartungswert sind

Chebyshev: Für eine Zufallsvariable X und ein beliebiges $t \in \mathbb{R}$ mit $t > 0$ gilt:

$$\Pr[|X - \mathbb{E}[X]| \geq t] \leq \frac{\text{Var}[X]}{t^2} \quad \text{oder anders umgeformt: } \Pr[|X - \mathbb{E}[X]| \geq t \cdot \underbrace{\sqrt{\text{Var}[X]}}_{= \text{Standardabweichung } \sigma}] \leq \frac{1}{t^2}$$

= Standardabweichung σ



Intuitiv: Es ist unwahrscheinlich, dass die Werte von X weiter als t Standardabweichungen vom Erwartungswert entfernt sind

Chernoff: Für $X = \sum_{i=1}^n X_i$, wobei $X_1, \dots, X_n$ unabhängige Zufallsvariablen sind mit $X_i \sim \text{Bernoulli}(p_i)$,

gilt:

1.) $\Pr[X \geq (1 + \delta) \cdot \mathbb{E}[X]] \leq e^{-\frac{1}{2} \delta^2 \cdot \mathbb{E}[X]}$ für alle $0 < \delta \leq 1$

2.) $\Pr[X \leq (1 - \delta) \cdot \mathbb{E}[X]] \leq e^{-\frac{1}{2} \delta^2 \cdot \mathbb{E}[X]}$ für alle $0 < \delta \leq 1$

3.) $\Pr[X \geq t] \leq 2^{-t}$ für alle $t \geq 2e \mathbb{E}[X]$

Aufgaben: (sehr wichtig, kommt vermutlich mehrmals bei der Prüfung dran)

Die Lebensdauer X in Stunden einer bestimmten Art von Glühbirne hat einen Erwartungswert von $\mathbb{E}[X] = 1000$.

Schätze $Pr[X \geq 2500]$ (also die Wahrscheinlichkeit, dass eine zufällig ausgewählte Glühbirne 2500 Stunden oder länger leuchtet) mit einer passenden Ungleichung ab.

$$Pr[X \geq 2500] \stackrel{\text{Markov}}{\leq} \frac{\mathbb{E}[X]}{2500} = \frac{1000}{2500} = 0.4 \quad (\text{Markov ist erlaubt weil } X \text{ nicht-negativ ist})$$

Eine Maschine füllt Kaffeebohnen in Säcke ab. Das Gewicht X der Säcke schwankt leicht. Es ist bekannt, dass der Erwartungswert $\mathbb{E}[X] = 500g$ beträgt und die Varianz $Var[X] = 16g^2$.

Schätze $Pr[|X - \mathbb{E}[X]| \geq 12]$ (also die Wahrscheinlichkeit, dass ein Sack um 12g oder mehr vom Erwartungswert abweicht) mit einer passenden Ungleichung ab.

$$Pr[|X - \mathbb{E}[X]| \geq 12] \stackrel{\text{Chebyshev}}{\leq} \frac{Var[X]}{12^2} = \frac{16}{12^2} = \frac{1}{9} \approx \underline{\underline{0.11}}$$

Die Temperatur T in einer Stadt kann zwischen $-10^\circ C$ und $35^\circ C$ schwanken. Der Erwartungswert liegt bei $\mathbb{E}[T] = 10$.

Schätze $Pr[T \geq 30]$ mit einer passenden Ungleichung ab (oder begründe, falls keine der Ungleichungen anwendbar ist).

Markov ist nicht auf T anwendbar, weil T negative Werte annehmen kann

Für Chebyshev fehlt die Varianz...

Idee: Sei $X := T + 10$ eine nicht-negative Zufallsvariable

$$\mathbb{E}[X] = \mathbb{E}[T + 10] \stackrel{\text{Linearität}}{=} \mathbb{E}[T] + 10 = 10 + 10 = 20$$

$$Pr[T \geq 30] \stackrel{\text{Markov}}{\leq} \frac{\mathbb{E}[X]}{40} = \frac{20}{40} = \underline{\underline{0.5}}$$

$\xrightarrow{\text{+10 auf beiden Seiten der Ungleichung}}$
 $\xrightarrow{X = T + 10}$

Die Anzahl der verkauften Croissants X pro Tag in einem Café hat einen Erwartungswert von $\mathbb{E}[X] = 20$ und eine bekannte Varianz von $\text{Var}[X] = 16$.

Schätze $\Pr[X \geq 30]$ (also die Wahrscheinlichkeit, dass an einem Tag 30 oder mehr Croissants verkauft werden) möglichst genau mit einer passenden Ungleichung ab.

mit Markov : $\Pr[X \geq 30] \stackrel{\text{Markov}}{\leq} \frac{\mathbb{E}[X]}{30} = \frac{20}{30} \approx \underline{\underline{0.67}}$

mit Chebyshev: $\Pr[X \geq 30] = \Pr[X - \mathbb{E}[X] \geq 30 - \mathbb{E}[X]] \quad (\mathbb{E}[X] \text{ auf beiden Seiten subtrahieren})$

$= \Pr[X - \mathbb{E}[X] \geq 30 - 20] \quad (\mathbb{E}[X] = 20)$

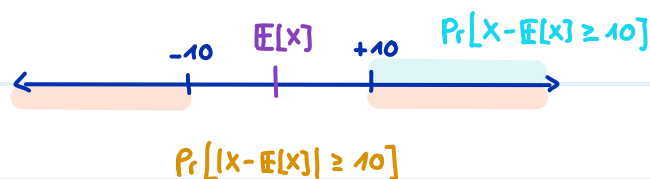
$= \Pr[X - \mathbb{E}[X] \geq 10]$

$\leq \Pr[|X - \mathbb{E}[X]| \geq 10]$

$\stackrel{\text{Chebyshev}}{\leq} \frac{\text{Var}[X]}{10^2}$

$= \frac{16}{100} = \underline{\underline{0.16}}$

Diese Ungleichung kommt sehr oft vor.



Ein Cloud-Dienst hat genau $n = 1000$ registrierte Nutzer. In einer bestimmten Minute greift jeder Nutzer unabhängig von den anderen mit einer Wahrscheinlichkeit von 10% auf den Server zu. Sei X die Anzahl der Nutzer, die in dieser Minute zugreifen. X ist eine Summe von unabhängigen Bernoulli Verteilungen, also $X \sim \text{Bin}(1000, 0.1)$.

Schätze $\Pr[X \geq 150]$ (also die Wahrscheinlichkeit, dass 150 oder mehr Nutzer gleichzeitig zugreifen) mit einer passenden Ungleichung ab.

$$\mathbb{E}[X] = n \cdot p = 1000 \cdot 0.1 = 100$$

$$\text{Var}[X] = n \cdot p \cdot (1-p) = 1000 \cdot 0.1 \cdot 0.9 = 90$$

$$\text{mit Markov: } \Pr[X \geq 150] \stackrel{\text{Markov}}{\leq} \frac{\mathbb{E}[X]}{150} = \frac{100}{150} \approx \underline{\underline{0.67}}$$

$$\begin{aligned} \text{mit Chebyshev: } \Pr[X \geq 150] &= \Pr[X - \mathbb{E}[X] \geq 150 - \mathbb{E}[X]] && (\mathbb{E}[X] \text{ auf beiden Seiten subtrahieren}) \\ &= \Pr[X - \mathbb{E}[X] \geq 150 - 100] && (\mathbb{E}[X] = 100) \\ &= \Pr[X - \mathbb{E}[X] \geq 50] \\ &\leq \Pr[|X - \mathbb{E}[X]| \geq 50] && (\text{gleiche Idee wie bei vorheriger Aufgabe}) \\ &\stackrel{\text{Chebyshev}}{\leq} \frac{\text{Var}[X]}{50^2} \\ &= \frac{90}{2500} = \underline{\underline{0.036}} \end{aligned}$$

$$\begin{aligned} \text{mit Chernoff: } \Pr[X \geq 150] &= \Pr\left[X \geq \frac{150}{\mathbb{E}[X]} \cdot \mathbb{E}[X]\right] && (\text{mit } \frac{\mathbb{E}[X]}{\mathbb{E}[X]} \text{ erweitern}) \\ &= \Pr\left[X \geq \frac{150}{100} \cdot \mathbb{E}[X]\right] && (\mathbb{E}[X] = 100) \\ &= \Pr\left[X \geq \left(1 + \frac{1}{2}\right) \mathbb{E}[X]\right] && (\text{weil } \frac{150}{100} = 1.5 = 1 + \frac{1}{2}) \\ &\stackrel{\text{Chernoff (1)}}{\leq} e^{-\frac{1}{3} \left(\frac{1}{2}\right)^2 \mathbb{E}[X]} && (\text{wobei } \delta = \frac{1}{2}) \\ &= e^{-\frac{1}{12} \cdot 100} \approx \underline{\underline{0.00024}} && (\mathbb{E}[X] = 100) \end{aligned}$$

Target Shooting

Gegeben: Zwei Mengen ein Subset S von einem Universum U

und eine Indikatorfunktion $I_S(u) = \begin{cases} 1 & \text{falls } u \in S \\ 0 & \text{sonst} \end{cases}$

Problemstellung: Schätze den relativen Anteil $\frac{|S|}{|U|}$

Algorithmus:

1. Wähle $u_1, \dots, u_N \in U$ zufällig
2. return $\frac{1}{N} \cdot \sum_{i=1}^N I_S(u_i)$ = Durchschnitt

- Sei $Y := \frac{1}{N} \sum_{i=1}^N I_S(u_i)$ (= Ausgabe des Algorithmus)
- $\mathbb{E}[Y] = \frac{1}{N} \sum_{i=1}^N \mathbb{E}[I_S(u_i)] = \frac{1}{N} \cdot N \cdot \frac{|S|}{|U|} = \frac{|S|}{|U|}$
 $\xrightarrow{\text{Linearität}} \quad \xrightarrow{\mathbb{E}[I_S(u_i)] = \frac{|S|}{|U|}}$

Wie gross muss N sein?

- Sei $N \geq 3 \frac{|U|}{|S|} \cdot \varepsilon^{-2} \ln(\frac{2}{\delta})$ für $\varepsilon, \delta > 0$. Dann ist die Ausgabe des Algorithmus mit Wahrscheinlichkeit $\geq 1 - \delta$ im Intervall $[(1-\varepsilon)\frac{|S|}{|U|}, (1+\varepsilon)\frac{|S|}{|U|}]$

Beweis

$$\begin{aligned}
 & \overbrace{\underbrace{Y \text{ zu klein}}_{P_r[Y < (1-\varepsilon)\frac{|S|}{|U|}]} + \underbrace{Y \text{ zu gross}}_{P_r[Y > (1+\varepsilon)\frac{|S|}{|U|}]}}^{Y \text{ nicht im Intervall}} \\
 & \leq P_r[Y \leq (1-\varepsilon)\frac{|S|}{|U|}] + P_r[Y \geq (1+\varepsilon)\frac{|S|}{|U|}] \\
 & = P_r[Y \leq (1-\varepsilon)\mathbb{E}[Y]] + P_r[Y \geq (1+\varepsilon)\mathbb{E}[Y]] \\
 & = P_r[NY \leq (1-\varepsilon)\mathbb{E}[NY]] + P_r[NY \geq (1+\varepsilon)\mathbb{E}[NY]] \\
 & \leq e^{-\frac{1}{2}\varepsilon^2 \mathbb{E}[NY]} + e^{-\frac{1}{2}\varepsilon^2 \mathbb{E}[NY]}
 \end{aligned}$$

↳ $<$ wird zu $\leq$, und $>$ wird zu $\geq$

↳ $\mathbb{E}[Y] = \frac{|S|}{|U|}$

↳ beide Seiten der Ungleichungen mal N

↳ (hernoff i) rechts und ii) links

$$\leq 2 e^{-\frac{1}{3}\varepsilon^2 \mathbb{E}[NY]}$$

$$= 2 e^{-\frac{1}{3}\varepsilon^2 N \frac{|S|}{|V|}}$$

↪ weil $e^{-\frac{1}{3}} > e^{-\frac{1}{2}}$

↪ $\mathbb{E}[NY] = N \mathbb{E}[Y] = N \cdot \frac{|S|}{|V|}$

Wir setzen $2 e^{-\frac{1}{3}\varepsilon^2 N \frac{|S|}{|V|}} \leq \delta$ und lösen nach N auf

$$\Leftrightarrow e^{-\frac{1}{3}\varepsilon^2 N \frac{|S|}{|V|}} \leq \frac{\delta}{2}$$

$$\Leftrightarrow -\frac{1}{3}\varepsilon^2 N \frac{|S|}{|V|} \leq \ln\left(\frac{\delta}{2}\right)$$

$$\Leftrightarrow N \geq -3 \frac{|V|}{|S|} \varepsilon^{-2} \ln\left(\frac{\delta}{2}\right)$$

$$\Leftrightarrow \underline{\underline{N \geq 3 \frac{|V|}{|S|} \varepsilon^{-2} \ln\left(\frac{2}{\delta}\right)}}$$

↪ $\cdot \frac{1}{2}$ auf beiden Seiten

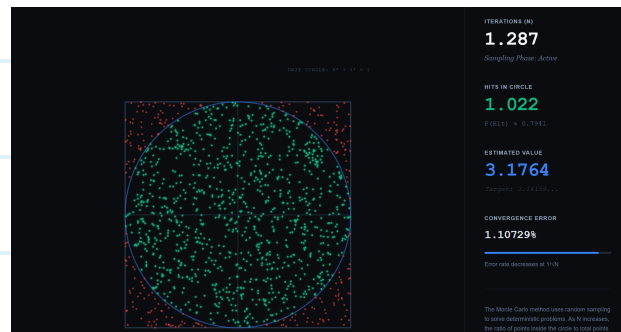
↪ $\ln(\cdot)$ auf beiden Seiten

↪ $\cdot -3 \frac{|V|}{|S|} \varepsilon^{-2}$, $\leq$ dreht sich um bei Multiplikation mit negativem Wert

↪ Logarithmusgesetz: $\ln\left(\frac{a}{b}\right) = \ln\left(\left(\frac{b}{a}\right)^{-1}\right) = -\ln\left(\frac{b}{a}\right)$

- solche Beweise verstehen, d.h. schrittweise nachvollziehen ist wichtig, aber man muss sie nicht auswendig können

Bsp.



→ Randomisierte Algorithmen

Monte-Carlo

vs

Las-Vegas

- Korrektheit ist eine Zufallsvariable

- "meistens" korrekt

- immer schnell

- Laufzeit ist eine Zufallsvariable

- "meistens" schnell

- immer korrekt

- äquivalente Definition: Algorithmus, der immer schnell ist, aber manchmal ??? ausgibt, und sonst immer korrekt ist

" $\Rightarrow$ "

normalen Las-Vegas Algorithmus nach konstanter Zeit abbrechen und ??? ausgeben falls das Ergebnis noch nicht gefunden wurde

" $\Leftarrow$ "

wiederholen, solange ??? ausgegeben wird

- kann in Monte-Carlo umgewandelt werden (nach konstanter Zeit abbrechen und raten falls das Ergebnis noch nicht gefunden wurde)

Aufgabe:

Entscheide ob es sich um einen Monte-Carlo oder einen Las-Vegas Algorithmus handelt

- Maximum Suche: Wähle ein zufälliges Element und prüfe ob es grösser gleich alle anderen ist. Falls ja, dann gib das Element aus, falls nein, wiederhole.

Las-Vegas

- Target Shooting, wobei wir eine Ausgabe als korrekt werten, falls sie im Intervall ist.

Monte-Carlo

- Target Shooting, wobei wir eine Ausgabe als korrekt werten, falls die Ausgabe exakt dem echten Verhältnis $\frac{|S|}{|U|}$ entspricht.

keines von beiden. z.B. π ist irrational, d.h. die Ausgabe wird nie exakt sein

- QuickSort mit zufälliger Pivot-Element Wahl

Las-Vegas

- Prüfe ob ein Graph ein Dreieck hat: Drei zufällige Knoten werden ausgewählt. Falls sie ein Dreieck bilden gib JA zurück, ansonsten gib NEIN zurück.

Monte - Carlo

- Suche einen Hamiltonkreis in einem Graphen in einem Hamiltonschen Graphen: Wähle eine zufällige Permutation der Knoten und prüfe ob zwischen allen Knoten eine Kante existiert. Falls ja, dann gib die Permutation aus, falls nein, wiederhole.

Las-Vegas

Fehlerreduktion

Für Las-Vegas:

- Sei A ein Las-Vegas Algorithmus der manchmal ??? ausgibt
- aktuelle Korrektheit $\cdot \Pr[A(I) \text{ ist korrekt}] \geq \varepsilon$
- Wir wählen den Zielfehler δ
- A_δ ruft A bis zu $N := \lceil \varepsilon^{-1} \ln(\delta^{-1}) \rceil$ mal auf
 - sobald A etwas anderes als ??? ausgibt, gibt A_δ dasselbe aus
 - sonst gibt A_δ am Ende ??? aus
- $\Pr[A_\delta(I) \text{ ist korrekt}] \geq 1 - \delta$
- Beweis: $\Pr[A_\delta(I) = ???] = \Pr[A(I) = ???]^N$

$$\leq (1 - \varepsilon)^N$$

$$\leq (e^{-\varepsilon})^N$$

$$\hookrightarrow \Pr[A(I) = ???] \geq 1 - \varepsilon$$

$$\hookrightarrow 1 + x \leq e^x \quad \forall x \in \mathbb{R}$$

hier mit $x = -\varepsilon$

$$\leq e^{-\varepsilon \varepsilon^{-1} \ln(\delta^{-1})}$$

$$= e^{-\ln(\delta^{-1})}$$

$$= e^{\ln(\delta)}$$

$$= \delta$$

$$\hookrightarrow \text{def } N = \lceil \varepsilon^{-1} \cdot \ln(\delta^{-1}) \rceil$$

$$\hookrightarrow \varepsilon \varepsilon^{-1} = 1$$

$$\hookrightarrow \text{Logarithmusgesetz: } \ln(x^{-1}) = -\ln(x)$$

$$\hookrightarrow e^x \text{ und } \ln(x) \text{ sind Umkehrfunktionen}$$

Für Monte-Carlo mit einseitigem Fehler:

- Sei A ein Monte-Carlo Algorithmus der immer korrekt ist für JA-Instanzen, aber nur mit Wahrscheinlichkeit $\geq \varepsilon$ korrekt ist für NEIN-Instanzen
- Wir wählen den Zielfehler δ
- A_δ ruft A bis zu $N := \lceil \varepsilon^{-1} \ln(\delta^{-1}) \rceil$ mal auf
 - sobald A einmal NEIN ausgibt, gibt A_δ auch NEIN aus
 - sonst gibt A_δ am Ende JA aus
- $\Pr[A_\delta(I) \text{ ist korrekt}] \geq 1 - \delta$

Für Monte-Carlo mit Fehlerwahrscheinlichkeit $< \frac{1}{2}$:

- Sei A ein Monte-Carlo Algorithmus der immer entweder JA oder NEIN ausgibt
- aktuelle Korrektheit $\cdot \Pr[A(I) \text{ ist korrekt}] \geq \frac{1}{2} + \varepsilon$
- Wir wählen den Zielfehler δ
- A_δ ruft A genau $N := \lceil 4 \cdot \varepsilon^{-2} \ln(\delta^{-1}) \rceil$ mal auf
 - A_δ gibt am Ende die Mehrheit der JA's und NEIN's aus
- $\Pr[A_\delta(I) \text{ ist korrekt}] \geq 1 - \delta$

Aufgaben:

Ein Programm sucht nach einem Weg durch ein Labyrinth. In 20% der Fälle findet es blitzschnell den korrekten Weg zum Ziel. In den restlichen 80% der Fälle verläuft es sich in einer Endlosschleife, bricht irgendwann ab und gibt die Meldung "???" aus. Wir möchten den Weg mit einer Sicherheit von 99% finden.

Können wir Fehlerreduktion anwenden? Wenn ja, wie oft müssen wir ihn aufrufen?

Fehlerreduktion für Las-Vegas: Korrektheit: $\varepsilon = 0.2$

Zielfehler: $\delta = 0.01$

$$N = \lceil \varepsilon^{-1} \cdot \ln(\delta^{-1}) \rceil = \lceil \frac{1}{0.2} \ln(\frac{1}{0.01}) \rceil = \lceil 5 \cdot \ln(100) \rceil = \underline{\underline{24}}$$

Ein Algorithmus überprüft, ob ein Computernetzwerk sicher ist. Wenn das Netzwerk sicher ist (JA), sagt der Algorithmus immer die Wahrheit. Wenn es eine Sicherheitslücke gibt (NEIN), erkennt der Algorithmus diese Lücke leider nur in 25% der Fälle. Wir möchten das Netzwerk absichern und eine maximale Irrtumswahrscheinlichkeit von 5% zulassen.

Können wir Fehlerreduktion anwenden? Wenn ja, wie oft müssen wir ihn aufrufen?

Fehlerreduktion für Monk-Carlo mit einseitigem Fehler:

Korrektheit für NEIN-Instanzen: $\varepsilon = 0.25$

Zielfehler: $\delta = 0.05$

$$N = \lceil \varepsilon^{-1} \cdot \ln(\delta^{-1}) \rceil = \lceil \frac{1}{0.25} \ln(\frac{1}{0.05}) \rceil = \lceil 4 \ln(20) \rceil = \underline{\underline{12}}$$

Ein Algorithmus versucht vorherzusagen, ob eine Aktie morgen steigt (JA) oder fällt (NEIN). Er ist extrem schlecht und liegt nur in 40% der Fälle richtig. Wir wollen eine Sicherheit von 95% erreichen, um unser Geld sicher anzulegen.

Können wir Fehlerreduktion anwenden? Wenn ja, wie oft müssen wir ihn aufrufen?

Fehlerreduktion für Monk-Carlo mit Fehlerwahrscheinlichkeit $< \frac{1}{2}$:

Idee: wir negieren die Ausgabe des Algorithmus: $\Pr[\overline{A(I)} \text{ ist korrekt}] = 0.6 = \frac{1}{2} + \varepsilon$ wobei $\varepsilon = 0.1$

Zielfehler: $\delta = 0.05$

$$N = \lceil 4 \varepsilon^{-2} \ln(\delta^{-1}) \rceil = \lceil 4 \frac{1}{0.1^2} \ln(\frac{1}{0.05}) \rceil = \lceil 4 \cdot 100 \ln(20) \rceil = \underline{\underline{1199}}$$

Idee: n ist eine Primzahl $\Rightarrow \text{ggT}(a, n) = 1 \quad \forall a \in \{1, 2, \dots, n-1\}$

EUKLID-PRIMZahlTEST(n)

- 1: Wähle $a \in [n - 1]$, zufällig gleichverteilt
 - 2: **if** $\text{ggT}(a, n) = 1$ **then return** 'Primzahl'
 - 3: **else return** 'keine Primzahl'
-
- Immer korrekt für JA-Instanzen (= Primzahlen)
 - Korrekt mit Wahrscheinlichkeit $\leq O\left(\frac{1}{\sqrt{n}}\right)$ im Worst Case für NEIN-Instanzen (= nicht-Primzahlen)
 - Monte-Carlo Algorithmus mit einseitigem Fehler
 - Laufzeit $O(\log(n)^3)$

DiskMat Recap.

$a \bmod b$ = Rest, wenn wir b so oft wie möglich von a subtrahieren und ≥ 0 bleiben

Bsp $10 \bmod 3 = 1$, $4 \equiv_2 0$, $7 \equiv_4 3$

$\mathbb{Z}_n = \{0, 1, \dots, n-1\}$ ist eine additive Gruppe mit der Operation $\oplus_n$
 $\hookrightarrow$ Assoziativität, neutrales Element und Inverse bezüglich Addition

$\mathbb{Z}_n^* = \{a \in [n-1] \mid \underbrace{gg^T(a, n) = 1}_{\text{stellt sicher dass es multiplikative Inversen gibt}}\}$ ist eine multiplikative Gruppe mit der Operation $\odot_n$

Lagrange: $\forall a \in \mathbb{Z}_n^*: a^{|\mathbb{Z}_n^*|} \equiv_n 1$

kleiner Fermatscher Satz: Falls p eine Primzahl ist, dann gilt für alle $a \in \{1, 2, \dots, p-1\}$:

$$a^{p-1} \equiv_p 1$$

FERMAT-PRIMZAHLTTEST(n)

- 1: Wähle $a \in [n-1]$, zufällig gleichverteilt
 - 2: **if** $a^{n-1} \bmod n = 1$ **then return** 'Primzahl'
 - 3: **else return** 'keine Primzahl'
-

- Immer korrekt für JA-Instanzen (= Primzahlen)
- Korrekt mit Wahrscheinlichkeit $> \frac{1}{2}$ für NEIN-Instanzen (= nicht-Primzahlen)
die keine Carmichael-Zahlen sind
- Ein **Zeuge/Zertifikat** ist ein $a \in \mathbb{Z}_n^*$, das bezeugt, dass n keine Primzahl ist
(also hier falls $a^{n-1} \not\equiv_n 1$)
- Eine **Carmichael-Zahl** ist eine zusammengesetzte Zahl, für die es aber keine Zeugen gibt
 $\Leftrightarrow n$ ist nicht prim aber $\forall a \in \mathbb{Z}_n^*$ gilt $a^{n-1} \equiv_n 1$
- a ist eine **Pseudoprimzahlbasis** von n , falls $a^{n-1} \equiv_n 1$ aber n trotzdem keine Primzahl ist
("Lügner")
- Fehlerreduktion nicht möglich (wegen Carmichael-Zahlen)

Idee: $\mathbb{Z}_n^*$ ist ein Körper $\Leftrightarrow n$ ist prim

und ein Polynom vom Grad 2 hat höchstens 2 Nullstellen in einem Körper

$\Rightarrow P(x) = x^2 - 1$ hat höchstens zwei Nullstellen

$\Rightarrow x^2 = 1$ hat höchstens zwei Lösungen, nämlich $x=1$ und $x=-1 \equiv_{n-1}$

$\Rightarrow$ Gegeben ein $a \in \{2, \dots, n-1\}$

falls $a^{n-1} \not\equiv_n 1$ dann ist n sicher keine Primzahl (Fermat)

falls $a^{n-1} \equiv_n 1$ dann machen wir weiter falls wir die Wurzel ziehen können.

falls $\sqrt{a^{n-1}} \notin \{1, n-1\}$ dann ist n sicher keine Primzahl

falls $\sqrt{a^{n-1}} \equiv_n n-1$ können wir nicht weiter machen

falls $\sqrt{a^{n-1}} \equiv_n 1$ dann machen wir weiter falls wir noch einmal die Wurzel ziehen können.

falls $\sqrt{\sqrt{a^{n-1}}} \notin \{1, n-1\}$ dann ist n sicher keine Primzahl
usw..

Formeller: 1) Sei also $n > 2$ prim, dann ist $\mathbb{Z}_n^*$ ein Körper.

2) Finde k und d , sodass $n-1 = 2^k d$ und d ungerade ist

3.) $\forall a \in \{2, \dots, n-1\} \forall 1 \leq i \leq k: a^{2^i d} \equiv_n 1 \Rightarrow a^{2^{i-1} d} \in \{1, n-1\}$

$$x^2 = 1 \Rightarrow \sqrt{x} \in \{1, n-1\}$$

4) Also entweder $\forall 0 \leq i \leq k: a^{2^i d} \equiv_n 1$

oder $\exists i \in \{0, \dots, k-1\}: a^{2^i d} \equiv_n n-1$

falls keiner dieser Fälle eintritt ist p sicher keine Primzahl

MILLER-RABIN-PRIMZAHLTTEST(n) ($n > 1$, ungerade!)

1: Schreibe $n-1 = 2^k d$ mit $d, k \in \mathbb{N}$, d ungerade

2: Wähle $a \in [n-1]$, zufällig gleichverteilt

3: if $a^d \bmod n = 1$ oder $\exists i \in \{0, \dots, k-1\}: a^{2^i d} \bmod n = n-1$

then return 'Primzahl' $\leftarrow$ educated guess weil die Zwischenschritte $1, 1, 1, 1, \dots$ sind

4: else return 'keine Primzahl' $\leftarrow$ immer korrekt

oder $1, 1, \dots, 1, n-1, ?, ?, \dots$ sind

Weil es noch nicht kompliziert genug ist, drehen wir die Reihenfolge um i

Falls $x^2 = 1$, dann muss x vor dem Quadrieren 1 oder $n-1$ gewesen sein

MILLER-RABIN-PRIMZAHLTTEST(n)

```

1: if  $n = 2$  then
2:   return 'Primzahl'  $\leftarrow$  immer korrekt  $\checkmark$ 
3: else if  $n$  gerade oder  $n = 1$  then
4:   return 'keine Primzahl'  $\leftarrow$  immer korrekt
5: Wähle  $a \in \{2, 3, \dots, n-1\}$  zufällig und
6: berechne  $k, d \in \mathbb{Z}$  mit  $n-1 = d2^k$  und  $d$  ungerade.
7:  $x \leftarrow a^d \pmod n$ 
8: if  $x = 1$  or  $x = n-1$  then
9:   return 'Primzahl'  $\leftarrow$  educated guess, weil die Zwischenschritte  $1, 1, 1, 1, \dots$  sind falls  $x = 1$ 
                                oder  $n-1, 1, 1, 1, \dots$  falls  $x = n-1$ 
10: repeat  $k-1$  mal
11:    $x \leftarrow x^2 \pmod n$ 
12:   if  $x = 1$  then
13:     return 'keine Primzahl'  $\leftarrow$  immer korrekt, weil  $x$  vorher nicht 1 oder  $n-1$  war
14:   if  $x = n-1$  then
15:     return 'Primzahl'  $\leftarrow$  educated guess, weil die Zwischenschritte  $?, ?, \dots, n-1, 1, 1, \dots$  sind
16: return 'keine Primzahl'  $\leftarrow$  immer korrekt, weil hier  $a^{n-1} \not\equiv_n 1$  ist

```

- Immer korrekt für JA-Instanzen (= Primzahlen)
- Korrekt mit Wahrscheinlichkeit $\geq \frac{3}{4}$ für NEIN-Instanzen (= nicht-Primzahlen)
- Laufzeit: $O(\text{poly}(\log n))$

Verteilung der Primzahlen: $\pi(x)$ zählt wie viele Primzahlen $\leq x$ es gibt Bsp: $\pi(10) = 4$

$$\underbrace{\frac{\pi(x)}{x}}_{\text{Anteil Primzahlen der ersten } x \text{ Zahlen}} \approx \frac{1}{\ln(x)} \quad \text{sogar:} \quad \lim_{x \rightarrow \infty} \frac{\frac{\pi(x)}{x}}{\frac{1}{\ln(x)}} = 1$$

wow!!!
absolut wunderschön!!!
unglaublich cool!!!

$$\Pr[\text{zufällige Zahl aus } \{1, 2, \dots, x\} \text{ ist prim}] = \frac{1}{\ln(x)} \quad \text{für } x \rightarrow \infty$$

Ist die Fehlerwahrscheinlichkeit bei Euklids Primzahltest kleiner für $n = 15$ oder $n = 16$?

Fehler genau dann wenn $\text{ggT}(a, n) = 1$

$$\mathbb{Z}_{15}^* = \{a \in \{1, \dots, 14\} \mid \text{ggT}(a, 15) = 1\} = \{1, 2, 4, 7, 8, 11, 13, 14\} \Rightarrow \text{Fehlerwahrscheinlichkeit} = \frac{8}{14}$$

$$\mathbb{Z}_{16}^* = \{a \in \{1, \dots, 15\} \mid \text{ggT}(a, 16) = 1\} = \{1, 3, 5, 7, 9, 11, 13, 15\} \Rightarrow \text{Fehlerwahrscheinlichkeit} = \frac{8}{15}$$

$\Rightarrow$ kleinere Fehlerwahrscheinlichkeit bei $n = 16$.

Bestimme ob $a = 4$ und $a = 2$ Zertifikate oder Pseudoprimzahlbasen sind bei Fermats Primzahltest für $n = 15$.

(Formel: $a^{n-1} \equiv_n 1$)

Für $a = 4$: $4^{15-1} = 4^{14} = (4^2)^7 = 16^7 \equiv_{15} 1^7 = \underline{1}$

$\Rightarrow a = 4$ ist eine Pseudoprimzahlbasis, Algorithmus gibt fälschlicherweise "Primzahl" aus

Für $a = 2$: $2^{15-1} = 2^{14} = 2^{12} \cdot 2^2 = (2^4)^3 \cdot 4 = 16^3 \cdot 4 \equiv_{15} 1^3 \cdot 4 = \underline{4} \quad (\neq 1)$

$\Rightarrow a = 2$ ist ein Zertifikat, Algorithmus gibt korrekt "keine Primzahl" aus

Was gibt der Miller-Rabin Primzahltest aus für $n = 13, a = 2$?

$$n-1 = 12 = 2^2 \cdot 3 \Rightarrow k=2, d=3$$

Schritt $i = 0$: $2^3 \equiv_{13} 8$ (teste $a^{2^i d} \bmod n$)

$8 \neq 1$ und $8 \neq 12 \Rightarrow$ weiter machen

Schritt $i = 1$: $2^{2^2} \equiv_{13} 64 \equiv_{13} 12$ (teste $a^{2^i d} \bmod n$)

$12 = 12 \checkmark \Rightarrow$ return "Primzahl" (korrekt)

Was gibt der Miller-Rabin Primzahltest aus für $n = 21, a = 2$?

$$n-1 = 20 = 2^2 \cdot 5 \quad \Rightarrow k=2, d=5$$

$$\text{Schritt } i = 0. \quad 2^5 \equiv_{21} 32 \equiv_{21} 11 \quad \left(\text{test } a^{2^0 d} \bmod n \right)$$

$11 \neq 1$ und $11 \neq 20 \Rightarrow$ weiter machen

$$\text{Schritt } i = 1. \quad 2^{2 \cdot 5} \equiv_{21} (2^5)^2 \equiv_{21} 11^2 \equiv_{21} 121 \equiv_{21} 16 \quad \left(\text{test } a^{2^1 d} \bmod n \right)$$

$16 \neq 1$ und $16 \neq 20$.

Schleife endet $\Rightarrow$ return "keine Primzahl" (korrekt)

Was gibt der Miller-Rabin Primzahltest aus für $n = 17, a = 3$?

$$n-1 = 16 = 2^4 \quad \Rightarrow k=4, d=1$$

$$\text{Schritt } i = 0. \quad 3^1 \equiv_{17} 3 \quad \left(\text{test } a^{2^0 d} \bmod n \right)$$

$3 \neq 1$ und $3 \neq 16 \Rightarrow$ weiter machen

$$\text{Schritt } i = 1. \quad 3^2 \equiv_{17} 9 \quad \left(\text{test } a^{2^1 d} \bmod n \right)$$

$9 \neq 1$ und $9 \neq 16 \Rightarrow$ weiter machen

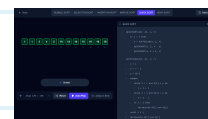
$$\text{Schritt } i = 2. \quad 3^{2^2} \equiv_{17} 3^4 \equiv_{17} 81 \equiv_{17} 13 \quad \left(\text{test } a^{2^2 d} \bmod n \right)$$

$13 \neq 1$ und $13 \neq 16 \Rightarrow$ weiter machen

$$\text{Schritt } i = 3. \quad 3^{2^3} \equiv_{17} (3^4)^2 \equiv_{17} 13^2 \equiv_{17} 169 \equiv_{17} 16 \quad \left(\text{test } a^{2^3 d} \bmod n \right)$$

$16 \stackrel{\vee}{=} 16 \Rightarrow$ return "Primzahl" (korrekt)

QuickSort mit zufälligem Pivot Element



- Las-Vegas Algorithmus mit erwarteter Laufzeit $O(n \ln n)$

Beweis: Sei $(a_1, a_2, \dots, a_n)$ das Array nach dem Sortieren

Seien a_i und a_j zwei beliebige Elemente, wobei $i < j$

Sei $I_{i,j} = \begin{cases} 1 & \text{falls } a_i \text{ mit } a_j \text{ verglichen wird} \\ 0 & \text{sonst} \end{cases}$ eine Indikatorvariable

Irgendwann wird das erste mal ein Pivot $p \in \{a_i, \dots, a_j\}$ ausgewählt.

Case $p \in \{a_i, a_j\}$: a_i und a_j werden einmal verglichen

Case $p \in \{a_{i+1}, \dots, a_{j-1}\}$: a_i und a_j werden nie verglichen, weil sie a_i im linken rekursiven Aufruf ist und a_j im rechten

$(a_1, \dots, a_{i-1}, \boxed{a_i, a_{i+1}, \dots, a_{j-1}, a_j}, a_{j+1}, \dots, a_n)$
 $j-i+1$ viele Elemente

$$\mathbb{E}[I_{i,j}] = \Pr[a_i \text{ wird mit } a_j \text{ verglichen}] = \frac{2}{j-i+1}$$

$$\mathbb{E}[\text{totale Anzahl Vergleiche}] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \mathbb{E}[I_{i,j}]$$

Summe über alle möglichen Paare $1 \leq i < j \leq n$

$$= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1}$$

$$\mathbb{E}[I_{i,j}] = \frac{2}{j-i+1}$$

$$= \sum_{i=1}^{n-1} \sum_{k=2}^{n-i+1} \frac{2}{k}$$

Index Substitution $k = j-i+1$

$$j=i+1 \Leftrightarrow j-i+1=2 \Leftrightarrow k=2$$

$$j=n \Leftrightarrow j-i+1=n-i+1 \Leftrightarrow k=n-i+1$$

$$\leq \sum_{i=1}^n \sum_{k=1}^n \frac{2}{k}$$

Summen gehen bis n

$$= 2 \cdot \sum_{i=1}^n \sum_{k=1}^n \frac{1}{k}$$

Distributivgesetz

$$= 2n \cdot H_n$$

Harmonische Reihe $H_n \stackrel{\text{def}}{=} \sum_{k=1}^n \frac{1}{k}$

$$\leq \underline{\underline{O(n \log n)}}$$

weil $H_n \leq \ln(n) + O(1)$

QuickSelect

Ziel: finde das k -kleinste Element in einem Array

QUICKSELECT(A, ℓ, r, k)

- 1: $p \leftarrow \text{Uniform}(\{\ell, \ell+1, \dots, r\})$ ▷ wähle Pivotelement zufällig
 - 2: $t \leftarrow \text{PARTITION}(A, \ell, r, p)$
 - 3: **if** $t = \ell + k - 1$ **then** ($k-1$ Elemente sind links von t)
 - 4: **return** $A[t]$ ▷ gesuchtes Element ist gefunden
 - 5: **else if** $t > \ell + k - 1$ **then**
 - 6: **return** QUICKSELECT($A, \ell, t-1, k$) ▷ gesuchtes Element ist links
 - 7: **else**
 - 8: **return** QUICKSELECT($A, t+1, r, k-t$) ▷ gesuchtes Element ist rechts
-

Erwartete Laufzeit: $O(n)$

(kam meines Wissens nach noch nie bei einer Prüfung)

Duplikate finden mit direktem Sortieren

Universum: U ist die Menge aller möglichen Elementen

Datensatz: $D = (s_1, s_2, \dots, s_n)$ ist eine Folge von n Elementen aus einem Universum U

Duplikat: Ein Paar (i, j) sodass $s_i = s_j$, wobei $i < j$

1.) Sortieren

2) Liste durchlaufen und dabei alle Duplikate ausgeben

Bsp $(4, 19, 1, 2, 4, 5, 2, 4)$ $\left. \begin{array}{l} \text{Index: } 1 \quad 2 \quad 3 \quad 4 \quad 5 \quad 6 \quad 7 \quad 8 \\ (1, 2, 2, 4, 4, 4, 5, 19) \end{array} \right\} \text{Sortieren}$

Duplikate: $(2, 3), (4, 5), (4, 6), (5, 6)$

Laufzeit: $O(\underbrace{n \log n}_{\text{Sortieren}} + \underbrace{|\text{Duplikate}|}_{\text{Duplikate ausgeben}})$

Wie viele Duplikate gibt es höchstens bei n Elementen? $\rightarrow \sum_{i=1}^{n-1} 1 = \frac{n(n-1)}{2}$
 $= \binom{n}{2}$

Duplikate finden mit Hashfunktion

Motivation: Vergleiche von "schwierigen" Elementen (z.B. Graphen, DNA-Strings) können teuer sein

Hashfunktion: $h: U \rightarrow \{1, \dots, m\}$ berechnet für ein Element $u \in U$ den Hashwert $h(u)$

- m soll viel kleiner als $|U|$ sein
- h soll effizient berechenbar sein
- h soll zufällig gleichverteilt sein. $\forall u \in U \forall i \in [m]: \Pr[h(u) = i] = \frac{1}{m}$
- h hat die Eigenschaft: $s_i = s_j \Rightarrow h(s_i) = h(s_j)$

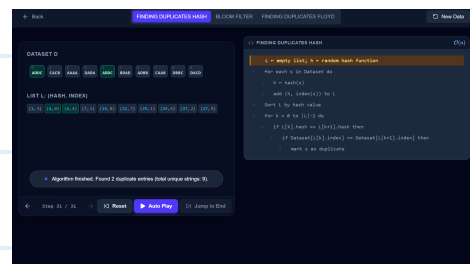
Kollisionen: sind Paare (i, j) mit $s_i \neq s_j$ aber $h(s_i) = h(s_j)$

1.) Hashen und Index merken

2.) Sortieren nach Hashwert aufsteigend

3.) Liste durchlaufen und dabei alle Hashwert-Duplikate

überprüfen und ausgeben, wenn es keine Kollisionen sind



Laufzeit: $O(\underbrace{n \log n}_{\text{Hashen und Sortieren}} + \underbrace{|\text{Duplikate}|}_{\text{Duplikate ausgeben}} + \underbrace{|\text{Kollisionen}|}_{\text{Kollisionen überprüfen}})$

Für $m = n^2$: ist die erwartete Anzahl Kollisionen konstant

Beweis: Sei $K_{i,j}$ die Indikatorvariable für das Ereignis " (i, j) ist eine Kollision"

$$\Rightarrow \Pr[K_{i,j} = 1] = \begin{cases} \frac{1}{m} & \text{falls } s_i \neq s_j \\ 0 & \text{sonst} \end{cases}$$

(weil $s_i = s_j$ ein Duplikat ist, und daher keine Kollision sein kann)

$$\Rightarrow \mathbb{E}[K_{i,j}] \leq \frac{1}{m}$$

$$\mathbb{E}[\text{Anzahl Kollisionen}] = \sum_{1 \leq i < j \leq n} \mathbb{E}[K_{i,j}]$$

$$\mathbb{E}[K_{i,j}] \leq \frac{1}{m}$$

$$\leq \sum_{1 \leq i < j \leq n} \frac{1}{m}$$

$$\binom{n}{2} = \text{Anzahl Paare } (i,j) \text{ mit } i < j$$

$$= \binom{n}{2} \cdot \frac{1}{m}$$

$$\text{def. Binomialkoeffizient } \binom{n}{k} = \frac{n!}{k!(n-k)!}$$

$$= \frac{n(n-1)}{2} \cdot \frac{1}{m}$$

$$\text{ausmultiplizieren}$$

$$= \frac{n^2 - n}{2} \cdot \frac{1}{m}$$

$$\text{Annahme: } m = n^2$$

$$= \frac{n^2 - n}{2} \cdot \frac{1}{n^2}$$

$$\text{dividieren}$$

$$= \frac{1 - \frac{1}{n}}{2}$$

$$= \frac{1}{2} - \frac{1}{2n}$$

$$< \frac{1}{2} \Rightarrow \text{Konstant}$$

Duplikate finden mit Hase-Igel-Algorithmus (Floyd)

Gegeben: ein Array $a[1..n]$ wobei alle Elemente zwischen 1 und $n-1$ liegen

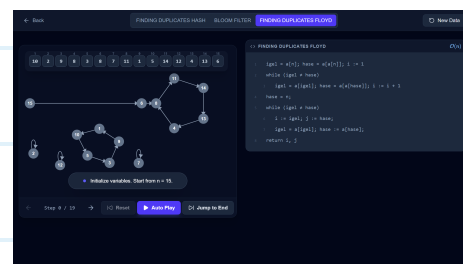
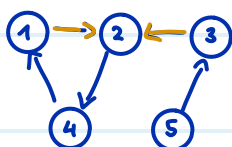
Gesucht: ein Duplikat, also zwei verschiedene Indices i, j sodass $a[i] = a[j]$

(existiert immer wegen Pigeonhole-Prinzip)

Umformulierung als Graph: Knoten $V = \{1, \dots, n\}$ und Kanten $\{(i, a[i]) \mid 1 \leq i \leq n\}$

neues Ziel: finde einen Knoten mit zwei eingehenden Kanten

Bsp. $A = (2, 4, 2, 1, 3)$

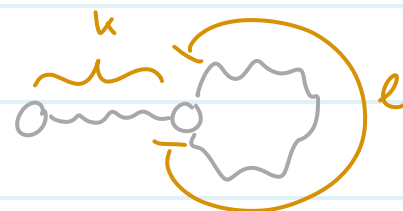


Subgraph: Wir betrachten nur noch alle von n aus erreichbaren Knoten.

Jeder Knoten hat genau eine ausgehende Kante (nämlich von i zu $a[i]$)

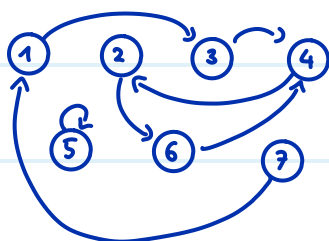
Der Subgraph muss ein Pfad der Länge $k \geq 1$ sein,

gefolgt von einem Kreis der Länge $\ell \geq 1$



Knotenfolge: $x_0 = n$, und rekursiv $x_{t+1} = a[x_t]$ für alle $t \geq 0$

Bsp $A = (3, 6, 4, 2, 5, 4, 1)$



$$x_0 = 7, x_1 = 1, x_2 = 3, x_3 = 4, x_4 = 2$$

$$x_5 = 6, x_6 = 4, x_7 = 2, x_8 = 6, \dots$$

1 Treffen: Es gibt ein $t \in \{1, \dots, n\}$ sodass $x_t = x_{2t}$

(also wenn Hase und Igel mit Geschwindigkeit 2 (Hase) und 1 (Igel) bei

Knoten n loslaufen, treffen sie sich spätestens nach n Schritten)

igel = $a[n]$; hase := $a[a[n]]$; $i := 1$

while (igel $\neq$ hase)

 igel = $a[\text{igel}]$; hase := $a[a[\text{hase}]]$; $i := i + 1$

1 Schritt

2 Schritte

Beweis: Sei r der "Rest" den wir auffüllen müssen, sodass $k+r$ das kleinstmögliche

Vielfache der Kreislänge ℓ ist: $k+r = \underbrace{\lceil \frac{k}{\ell} \rceil}_{\text{Anzahl angefangene male, wie oft } \ell \text{ in } k \text{ hinein passt}} \cdot \ell \Leftrightarrow r = \lceil \frac{k}{\ell} \rceil \cdot \ell - k$

Anzahl angefangene male, wie oft ℓ in k hinein passt

$\Rightarrow$ Dann liegt x_{k+r} auf dem Kreis (weil der Kreis bei k beginnt und $k+r \geq k$)

$\Rightarrow x_{k+r} = x_{2(k+r)}$ weil $k+r$ ein Vielfaches der Kreislänge ist. Also $x_{2(k+r)}$ macht einfach einige Kreisumrundungen mehr)

$\Rightarrow$ wir wählen also $t = k+r$

Und es gilt: $t = k+r$

$$= \lceil \frac{k}{\ell} \rceil \ell$$

per Definition

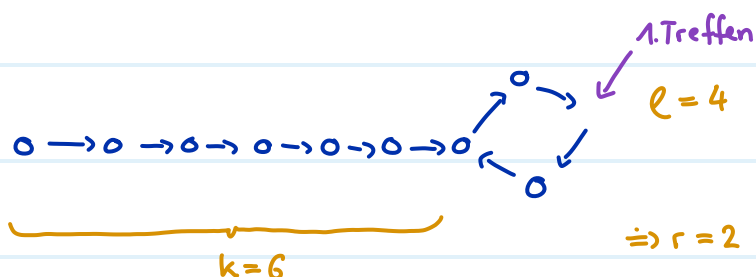
$$< \frac{k}{\ell} \ell + \ell$$

auf runden

$$= k + \ell$$

Pfadlänge + Kreislänge $\leq n$ weil es nur n Kanten gibt

$$\leq n$$



2 Treffen: es gilt: $\underbrace{x_k}_{\text{Hase}} = \underbrace{x_{k+t}}_{\text{Igel}}$

(der Hase startet noch einmal bei Knoten n und hat nun Geschwindigkeit 1. Er wird den Igel genau nach k Schritten treffen, also beim Anfang des Kreises)

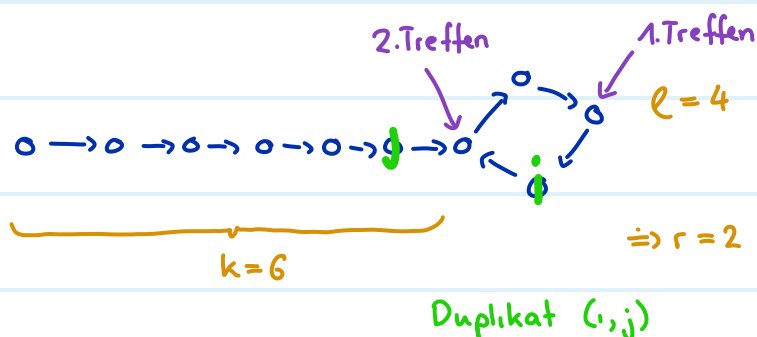
```

hase = n;   Hase wird zurück an den Startknoten  $n$  gestellt
while (igel  $\neq$  hase)
    i := igel; j := hase;
    igel = a[igel]; hase := a[hase];
return i, j
           1 Schritt           1 Schritt
  
```

Beweis: $x_{k+t} = x_k$ weil x_k auf dem Kreis liegt (x_k ist per Definition der erste Knoten des Kreises)
und t ein Vielfaches der Kreislänge ℓ ist (nach jeder Kreisumdrehung sind wir wieder bei x_k)

Laufzeit: $O(n)$

Zusätzlicher Speicher: 4 Variablen



Long Path Problem

Gegeben: ein Graph G und eine Länge B

Gesucht: gibt es einen Pfad der Länge B ?

hat also B Kanten und
 $B+1$ verschiedene Knoten

○—○—○—○—○

$B=4$ Kanten und

$B+1=5$ Knoten

Schwierigkeit: Falls das Hamiltonkreisproblem NP-schwer ist, dann ist auch Long-Path NP-schwer

Beweis: mittels Kontraposition ($A \rightarrow B \equiv \neg B \rightarrow \neg A$)

Annahme: Long-Path ist in polynomieller Zeit lösbar

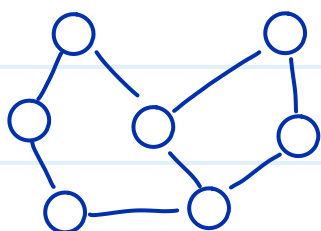
Zu zeigen: Hamiltonkreisproblem ist in polynomieller Zeit lösbar

Sei G ein beliebiger Graph mit n Knoten

Wir konstruieren G' wie folgt:

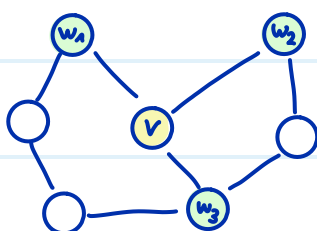
- Sei G' eine Kopie von G , ausser dass wir einen beliebigen Knoten v entfernen
- Dann erstelle für alle Kanten $\{v, w_i\}$ aus G einen neuen Knoten $\hat{w}_i$
und eine neue Kante $\{w_i, \hat{w}_i\}$ in G'
- G' hat also $\underbrace{(n-1)}_{\text{alle ausser } v} + \underbrace{\deg(v)}_{\text{für jede Kante von } v \text{ gibt es einen neuen Knoten } \hat{w}_i, \text{ und } \deg(v) \leq n-1} \leq 2n-2$ viele Knoten

G :



(G hat Hamiltonkreis)

wähle v

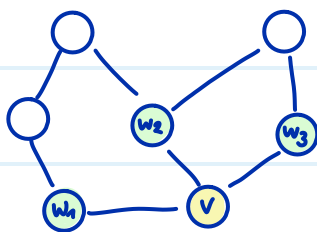


G' :

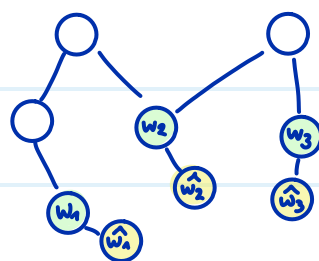


(G' hat Pfad der Länge n)

wähle v (andere Möglichkeit)



G' :



(G' hat Pfad der Länge n)

Zu zeigen: G hat einen Hamiltonkreis $\Leftrightarrow G'$ hat einen Pfad der Länge n

" $\Rightarrow$ " Annahme: $(v_1, v_2, \dots, v_n, v_1)$ ein Hamiltonkreis in G

Sei $v = v_1$ w.l.o.g. (wir können den Kreis rotieren sodass $v = v_1$)

$\Rightarrow (\hat{v}_2, v_2, v_3, \dots, v_n, \hat{v}_n)$ ist ein Pfad der Länge n in G'

$\underbrace{\hat{v}_2, v_2}_{v_2 \text{ war Nachbar von } v, \text{ deshalb gibt es } \hat{v}_2}$

$\underbrace{v_n, \hat{v}_n}_{v_n \text{ war Nachbar von } v \text{ und, deshalb gibt es } \hat{v}_n}$

" $\Leftarrow$ " Annahme: $(u_0, u_1, \dots, u_n)$ ist ein Pfad der Länge n in G'

$\Rightarrow$ die Knoten $u_1, \dots, u_{n-1}$ haben alle Grad ≥ 2 (weil sie innere Knoten im Pfad sind)

$\Rightarrow$ die Knoten $u_1, \dots, u_{n-1}$ sind genau die $n-1$ überlebenden Knoten aus G

(weil alle anderen Knoten $\hat{w}_i$ Grad 1 haben)

$\Rightarrow u_0 = \hat{w}_i$ und $u_n = \hat{w}_j$ sind zwei der neuen Knoten in G' (einzige übrige Möglichkeit)

$\Rightarrow u_1$ und u_{n-1} haben eine Kante zu v in G (weil sie eine Kante zu $\hat{w}_i / \hat{w}_j$ in G' haben)

$\Rightarrow (v, u_1, u_2, \dots, u_{n-1}, v)$ ist ein Hamiltonkreis in G

Schlussfolgerung: Falls Long-Path in polynomieller Zeit $O(t(n))$ entscheidbar ist, (t ist irgendeine funktion)

dann ist auch das Hamiltonkreisproblem in polynomieller Zeit $O(t(2n-2) + n^2)$

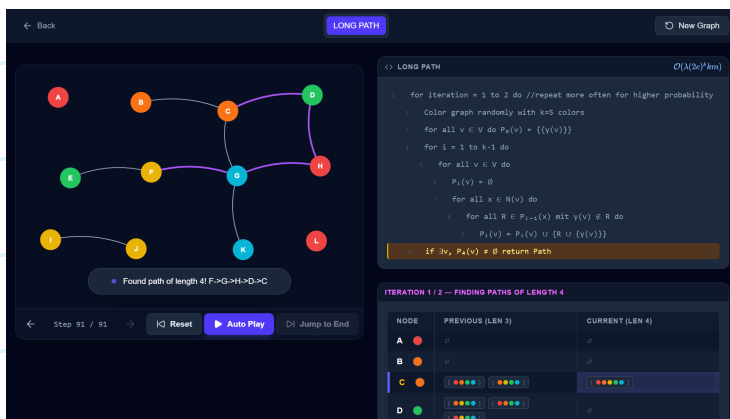
entscheidbar.

Long-Path
auf G'

Konstruktion
von G'

Monte Carlo Long Path

Gesucht: gibt es einen Pfad der Länge $k-1$? (also bestehend aus k Knoten)



1.) Färbe die Knoten mit einer zufällig gleichverteilten Färbung $\gamma: V \rightarrow [k]$

$\underbrace{k}_{k \text{ viele Farben}}$

• ein Pfad heisst **bunt** wenn alle Farben seiner Knoten paarweise verschieden sind

• Falls ein Pfad P der Länge $k-1$ existiert, dann existiert mit Wahrscheinlichkeit $\geq e^{-k}$

auch ein bunter Pfad der Länge $k-1$

Beweis: $\Pr[\exists \text{ bunter Pfad der Länge } k-1] \geq \Pr[\text{Pfad } P \text{ ist bunt}]$

$$= \frac{k^1}{k^k} \rightarrow \begin{array}{l} \text{Anzahl bunte Färbungen} \rightarrow \text{ziehen ohne zurücklegen} \\ \text{Anzahl aller Färbungen} \rightarrow \text{ziehen mit zurücklegen} \end{array}$$

$$\geq e^{-k}$$

$$\text{weil } e^k = \sum_{n=0}^{\infty} \frac{k^n}{n!} = 1 + k + \frac{k^2}{2!} + \frac{k^3}{3!} + \dots$$

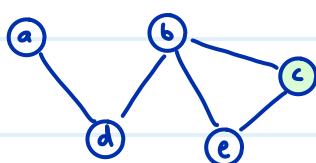
$$\Rightarrow e^k \geq \frac{k^k}{k^1} \quad \text{alles weglassen ausser } k\text{-ten Term}$$

$$\Rightarrow e^{-k} \leq \frac{k^1}{k^k} \quad \text{beide Seiten hoch } -1, \text{ Vergleichszeichen dreht sich um}$$

2.) DP Algorithmus bestimmt (mit absoluter Sicherheit) ob es einen Pfad der Länge $k-1$ gibt

Teilproblem: $P_i(v) = \left\{ \underbrace{S \subseteq [k]}_{\substack{S \text{ ist eine Menge} \\ \text{von } i+1 \text{ Farben}}} \mid |S|=i+1 \text{ und } \exists \text{ ein in } v \text{ endender, genau mit den} \right. \\ \left. \text{Farben aus } S \text{ gefärbter bunter Pfad der Länge } i \right\}$

Bsp



Farblegende: 1 = gelb, 2 = grün, 3 = rot, 4 = violett

Bestimme $P_1(c) = \{ \{2, 3\}, \{2, 1\} \}$

$P_2(a) = \{ \{4, 2, 3\} \}$

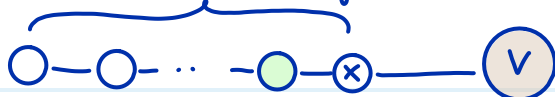
$P_2(b) = \{ \{3, 2, 4\}, \{3, 2, 1\} \}$

$P_0(e) = \{ \{1\} \}$

Base Case: $P_0(v) = \{ \{g(v)\} \}$ für alle Knoten $v \in V$

Rekursion: $P_i(v) = \bigcup_{\substack{x \in N(v) \\ \text{x muss ein} \\ \text{Nachbar von v sein,} \\ \text{damit wir den Pfad} \\ \text{verlängern können}}} \left\{ \underbrace{R \cup \{g(v)\}}_{\substack{\text{Farbset des} \\ \text{verlängerten Pfades}}} \mid \underbrace{R \in P_{i-1}(x)}_{\substack{\exists \text{ ein bunter} \\ \text{in x endender} \\ \text{Pfad mit genau} \\ \text{den Farben aus R} \\ \text{der Länge } i-1}} \text{ und } \underbrace{g(v) \notin R}_{\substack{\text{die Farbe von v wurde vorher} \\ \text{noch nicht verwendet}}} \right\}$

muss ein bunter Pfad der Länge $i-1$ sein



$R = \{ \dots, \text{green}, \dots \}$ und $\text{brown} \notin R$

$R \cup \{g(v)\} = \{ \dots, \text{green}, \text{brown} \}$

ITERATION 2 / 2 — FINDING PATHS OF LENGTH 3

NODE	PREVIOUS (LEN 2)	CURRENT (LEN 3)
A	[red, green]	[red, green, blue]
B	[red, green]	
C	[red, green]	
D	[red, green]	[red, green, blue]
E	[red, green]	[red, green, blue]
F	[red, green]	[red, green, blue]
G	[red, green]	[red, green, blue]
H	[red, green]	[red, green, blue]
I	[red, green]	[red, green, blue]
J	[red, green]	[red, green, blue]
K	[red, green]	[red, green, blue]
L	[red, green]	[red, green, blue]

Lösung: $\exists$ bunter Pfad der Länge $k-1 \Leftrightarrow \bigcup_{v \in V} P_{k-1}(v) \neq \emptyset \quad (\Leftrightarrow \exists v \in V: P_{k-1}(v) \neq \emptyset)$

Laufzeit: Base case in $O(n)$, danach wird $\text{Bunt}(G, i)$ $k-1$ mal ausgeführt

BUNT(G, i) macht den nächsten Rekursionsschritt

1: for all $v \in V$ do

2: $P_i(v) \leftarrow \emptyset$

3: for all $x \in N(v)$ do es gibt genau $\deg(v)$ viele Nachbarn

4: for all $R \in P_{i-1}(x)$ mit $\gamma(v) \notin R$ do

5: $P_i(v) \leftarrow P_i(v) \cup \{R \cup \{\gamma(v)\}\}$

$|R| = i$ viele Elemente überprüfen und kopieren

$$O(\text{Bunt}(G, i)) = O\left(\sum_{v \in V} \deg(v) \cdot |P_{i-1}(v)| \cdot 1\right)$$

$$\hookrightarrow |P_{i-1}(v)| \leq \binom{k}{i} \\ \text{weil } P_{i-1}(v) \subseteq \binom{[k]}{i}$$

$$\leq O\left(\sum_{v \in V} \deg(v) \cdot \binom{k}{i} \cdot 1\right)$$

$$\leq O\left(2m \cdot \binom{k}{i} \cdot 1\right)$$

$$\hookrightarrow \text{Handschlag-Lemma } \sum_{v \in V} \deg(v) = 2m$$

$\hookrightarrow 2$ weglassen

$$\leq O\left(\binom{k}{i} \cdot m \cdot 1\right)$$

$$\text{Total: } O\left(\sum_{i=1}^{k-1} \binom{k}{i} m \cdot i\right)$$

$$\hookrightarrow i \leq k$$

$$\leq O\left(\sum_{i=1}^{k-1} \binom{k}{i} m k\right)$$

$\hookrightarrow$ Summe von 0 bis k statt 1 bis k-1

$$\leq O\left(\sum_{i=0}^k \binom{k}{i} m k\right)$$

$$\hookrightarrow \sum_{i=0}^k \binom{k}{i} = 2^k \quad \text{folgt aus dem Binomialsatz, der aber für diesen Kurs nicht weiter relevant ist}$$

$$= \underline{\underline{O(2^k m k)}}$$

Bisher: Monte-Carlo Algorithmus mit einseitigem Fehler

- Immer korrekt für NEIN-Instanzen:

$$\Pr[\text{kein Pfad der Länge } k-1 \text{ wird gefunden} \mid \nexists \text{ Pfad der Länge } k-1] = 1$$

- Mit Wahrscheinlichkeit $\geq e^{-k}$ korrekt für JA-Instanzen:

$$\Pr[\text{Pfad der Länge } k-1 \text{ wird gefunden} \mid \exists \text{ Pfad der Länge } k-1] \geq e^{-k}$$

3.) Fehlerreduktion: Sei $\delta = e^{-\lambda}$ unser Zielfehler.

$\varepsilon = e^{-k}$ ist die aktuelle Korrektheit

$$\text{Anzahl Wiederholungen: } N = \lceil \varepsilon^{-1} \ln(\delta^{-1}) \rceil = \lceil e^k \ln(e^\lambda) \rceil = \underline{\underline{\lceil \lambda e^k \rceil}}$$

$\xrightarrow{\text{\tiny \(\varepsilon\ und \(\delta\) einsetzen}}}$
 $\xrightarrow{\text{\tiny \(\ln(e^\lambda) = \lambda\)}}}$

$$\text{Laufzeit insgesamt } O(\underbrace{2^k m k}_{\text{\tiny DP}} \cdot \underbrace{\lambda e^k}_{\text{\tiny Wiederholungen}}) = \underline{\underline{O(\lambda (2e)^k k m)}}$$

Für $k \leq O(\log n)$ ist die Laufzeit polynomiell:

$$O(\lambda (2e)^{\log(n)} \log(n) m) = O(\lambda n^{\log(2e)} \log(n) m)$$

weil $a^{\log b} = b^{\log a}$
 $\log(2e)$ ist konstant